

Fundamentals of Operating System Design

Module 4: System Calls

Prashant Jadav
DEPSTAR-CHARUSAT

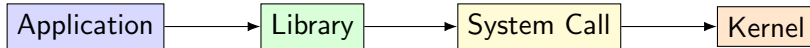
Learning Outcomes

- Define a system call.
- Explain why system calls are required.
- Identify common categories of system calls.
- Distinguish between application, library function and system call.

What is a System Call?

Definition

A system call is the interface through which a user program requests a service from the operating system kernel.



Why are System Calls Needed?

- Hardware protection
- Secure access to resources
- Controlled communication with the kernel
- Device independence
- Standard interface for applications

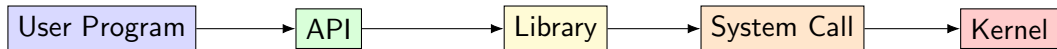
Categories of System Calls

- 1 Process Control
- 2 File Management
- 3 Device Management
- 4 Information Maintenance
- 5 Communication
- 6 Protection

Examples of Linux System Calls

System Call	Purpose
open()	Open a file
read()	Read data
write()	Write data
close()	Close file
fork()	Create process
exec()	Execute program
wait()	Wait for child
exit()	Terminate process

Execution Flow



Example Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Hello");
```

```
    return 0;
```

```
}
```

printf() eventually invokes the write() system call.

System Call vs Library Function

Library Function	System Call
Runs in user space	Runs in kernel space
Provided by C library	Provided by OS kernel
Portable	OS dependent
Example: <code>printf()</code>	Example: <code>write()</code>

Summary

- System calls provide controlled access to kernel services.
- Applications do not access hardware directly.
- Library functions often invoke system calls.
- System calls improve security and portability.

Quick Review

- ① What is a system call?
- ② Why are system calls required?
- ③ Give four examples of Linux system calls.
- ④ Differentiate a library function and a system call.